C200 Programming Assignment №6

Professor M.M. Dalkilic
Computer Science
School of Informatics, Computing, and Engineering

Indiana University, Bloomington, IN, USA

March 2, 2024

Introduction

Due Date: 11:00 PM, Friday, March 8, 2024 Please note that both Autograder submission `https://c200.luddy.indiana.edu/` and pushing to GitHub **must be done before the deadline. We do not accept late submissions**. As shared via Inscribe/Canvas, we will be checking for plagiarism. If this results in a grade change, we are compelled by the University to send a formal notification to the Dean.

Instructions

1. Make sure that you are **following the instructions** in the PDF, especially the format of output returned by the functions. For example, if a function is expected to return a numerical value, then make sure that a numerical value is returned (not a list or a dictionary). Similarly, if a list is expected to be returned then return a list (not a tuple, set or dictionary).

2. Test **debug** the code well (syntax, logical and implementation errors), before submitting to the Autograder. These errors can be easily fixed by running the code in VSC and watching for unexpected behavior such as, program failing with syntax error or not returning correct output.

3. Make sure that the **code does not have infinite loop (that never exits) or an endless recursion (that never completes)** before submitting to the Autograder. You can easily check for this by running in VSC and watching for program output, if it terminates timely or not.

4. Given that you already tried points 1-3, if you see that Autograder does not do anything (after you press 'submit') and waited for a while (30 seconds to 50 seconds), try refreshing the page or using a different browser.

5. Once you are done testing your code, comment out the tests i.e. the code under the __name__ == "__main__" section.

Problem 1: Recursion on Numbers

In this problem, you'll implement the following recursive functions.

$$p(0) = 10000 \tag{1}$$
$$p(n) = p(n-1) + 0.02p(n-1) \tag{2}$$

$$c(1) = 9 \tag{3}$$
$$c(n) = 9c(n-1) + 10^{n-1} - c(n-1) \tag{4}$$

$$d(0) = 1 \tag{5}$$
$$d(n) = 3d(n-1) + 1 \tag{6}$$

$$f(0) = 1 \tag{7}$$
$$f(1) = 1 \tag{8}$$
$$f(n) = f(n-1) + f(n-2) \tag{9}$$

$$e(1) = 12 \tag{10}$$
$$e(n) = -5e(n-1) \tag{11}$$

$$M(0,i) = 1 \; when \; c = 0 \tag{12}$$
$$M(c,i) = \begin{cases} 0 & c < 0 \;\vee\; f(i) > 10 \\ M(c - f(c), i+1) + M(c, i+1) & otherwise \end{cases}$$
$$\binom{n}{k} = \begin{cases} 1 & k = 0 \;\vee\; n = k \\ \binom{n-1}{k-1} + \binom{n-1}{k} & otherwise \end{cases} \tag{13}$$

Programming Problem 1: Recursion on Numbers

- Complete each of the functions.
- Observe that M requires f
- Note that $\binom{n}{k}$ indicates choosing k from n and in the starter code it is denoted by the function c_2(n, k).
- You can only use recursion–no math functions.

Problem 2: Recursion Over Other Types

The function f1 takes two strings $s_0 s_1 \cdots s_n, c_0 c_1 \cdots c_m$ defined

$$f1(\text{“”}, c_0 c_1 \cdots c_m) = \text{“”} \tag{14}$$

$$f1(s_0 s_1 \cdots s_n, c_0 c_1 \cdots c_m) = \begin{cases} \text{“1”} + f1(s_1 \cdots s_n, c_0 c_1 \cdots c_m) & s_0 \in c_0 c_1 \cdots c_m \\ \text{“0”} + f1(s_1 \cdots s_n, c_0 c_1 \cdots c_m) & o.w. \end{cases} \tag{15}$$

The function f2 takes a list of numbers:

$$f2([]) = [] \tag{16}$$

$$f2([x, \ldots]) = \log_2(x) + f2([\ldots]) \tag{17}$$

The function f3 takes a list and two objects x,y and returns True if x,y occur the same number of times in the list and False otherwise. Here are some runs:

```
data_4 = [["abe432","e2"],["1010","1"],["A man, a plan a canal--Panama.", "aA "]]
for d in data_4:
    print(f1(*d))

data_4 = [[1],[1,2],[1,2,1],[1,1,2],[2,1,1],[2,2,1,1]]
for d in data_4:
    print(f2(d))

data_43 = [([1,2,1,3,2],1,2),([1,2,1,3,2],1,3)]
for d in data_43:
    print(f3(*d))
```

has output

```
001001
1010
110100111001011101010000101010
0.0
1.0
1.0
1.0
1.0
2.0
True
False
```

Programming Problem 2: Recursion Over Other Types

- Complete each of the functions.
- *hint:* Function f3 will need to have a helper function with counters.

Problem 3: Pyramids of Egypt

This recursion is an old mathematical way to build a pyramid of numbers.

$$
\begin{aligned}
m([x]) &= [\,] && (18)\\
m([x,y]) &= [[x+y]] && (19)\\
m([x_0,x_1,x_2,\ldots,x_n] &= m([x_0+x_1,x_1+x_2,+\cdots,x_{n-1}+x_n]) + && (20)\\
& \quad [[x_0+x_1,x_1+x_2,\cdots,x_{n-1}+x_n]] && (21)
\end{aligned}
$$

The code:

```
x = [[1,2,3,4,5],[1],[3,4],[5,10,22],[1,2,3,4,5,6]]
for i in x:
    print(m(i))
```

produces

```
[[48], [20, 28], [8, 12, 16], [3, 5, 7, 9]]
[]
[[7]]
[[47], [15, 32]]
[[112], [48, 64], [20, 28, 36], [8, 12, 16, 20], [3, 5, 7, 9, 11]]
```

Observe, for example, [1,2,3,4,5]. This becomes the base: [1+2, 2+3,3+4,4+5]. Then this becomes the first story [3+5,5+7,7+9].

Deliverables Programming Problem 3

- Complete the function m

Problem 4: Caeser Cipher

The Caeser Cipher (CC) is a means of encoding a message. Please visit
`https://en.wikipedia.org/wiki/Caesar_cipher`
to read more about it (though it's not as complete as one would hope). We have decided to encode messages for this class using the CC, but with a slight modification. We'll only use the lower case alphabet, and we'll include space as an actual symbol. If you look at the character after 'z' in ASCII, you'll see the left brace "{".

```
>>> for i in range(ord('x'), ord('x') + 4):
...     f"{i} is {chr(i)} in ASCII"
...
'120 is x in ASCII'
'121 is y in ASCII'
'122 is z in ASCII'
'123 is { in ASCII'
>>>
```

We are including "{" in our alphabet after z as the symbol for space. You'll write two functions, encode(msg, shift) and decode(msg, shift) that encode a message and decode the message using shift as discussed in the wikipedia page, except our alphabet is one symbol larger. The following code:

```
data = ["abc xyz","the cat", "i love ctwohundred"]
for i,j in enumerate(data,start=2):
    print(f"original msg {j}")
    print(f"encoded  msg {encode(j,i)}")
    print(f"decoded  msg {decode(encode(j,i),i)}")

secret_msg = encode("the quick brown fox jumps over the lazy dog", 24)
print(secret_msg)
print(decode(secret_msg,24))
```

produces:

```
original msg abc xyz
encoded  msg cdebz{a
decoded  msg abc xyz
original msg the cat
encoded  msg wkhcfdw
decoded  msg the cat
original msg i love ctwohundred
encoded  msg mdpszidgx{slyrhvih
decoded  msg i love ctwohundred
```

```
qebxnrf{hxzoltkxcluxgrjmpxlsboxqebxiywvxald
the quick brown fox jumps over the lazy dog
```

Our output is obviously different from the wikipedia example, since we're expanding our alphabet to consider space a letter and including that in the shift.

Deliverables Programming Problem 4

- Complete the functions.
- You can use any normal string function that is covered in the class.

Problem 5: Towards Optimization of Stock Porfolio

In the programming portion of the midterm we had a part of a stock tracker, but really it's a **porfolio** which is a collection of financial instruments that can be bought and sold. Further in the class we'll look at *optimizing* the portfolio (yes, seems to be related to AI). A characterization of a portfolio we'll use is a bit older, but useful. A stock market can be represented as $\mathbf{X} = (X_1, X_2, \ldots, X_n)$ where X_i represents the ratio of the price at the end of the day to the price at the beginning of the day: $\frac{X_{end}}{X_{start}}$. So, for example, if $X_{end} = 109$ and $X_{start} = 92$, then $X = 1.18$ which means it's worth 18% more. A portfolio $\mathbf{b}$ is a distribution of a fund across stocks $(b_1, b_2, \ldots, b_n)$ where $b_i \geq 0, \sum b_i = 1$. For this problem we'll simplify this structure, but at its core, it's the same. We only keep b_i if we have purchased the stock and we won't require the fund to be completely dispersed. Here is the data:

```
stock = {0: {'A':200, 'B':20, 'C':125, 'D':90},
         1: {'A':190, 'B':25, 'C':122, 'D':91},
         2: {'A':195,  'B':30,'C':122, 'D':80}}

account = {'fund':1000}
```

The stock is a dictionary with a day: {name:price per share,...}. For example, day = 1 has B stock at $25 per share. The account begins with one entry, the funds available. You will implement two functions buy and sell.

The **buy** function takes day, stock name, an account, and number of shares to buy. The days will always be sequential: 0,1,2. The behavior is:

- Given the particular day, if sufficient funds exists, add to the account the key,value pair name:shares. If the key already exists, update the shares. Return the four tuple (w,x,y,z) where w is the stock name, x are the number of shares purchased, y is the price per share, and z is the total spent.
- If insufficient funds exist, return False without changing the account.

The **sell** function takes a day, stock name, an account, and number of shares. The behavior is:

- If the account has that stock, sell the shares, updating the funds.
  - If the shares are greater than or equal to the number of shares in the account, sell them all. Delete the key from account.
  - If the shares are less than the number of shares in the account, update that value.
  - Return the four tuple (w,x,y,z) where w is the stock name, x are the number of shares sold, y is the price per share on that day, and z is the total amount earned.

Here is a run:

```python
day = 0
print(f"day = {day}")
print(account)

print(buy(day, 'A',account,3))
print(account)
print(buy(day,'B',account,4))
print(account)
print(buy(day, 'B',account,1))
print(account)
print(buy(day,'D',account,4))
print(account)
day = 1
print(f"day = {day}")
print(sell(day,'A',account,2))
print(account)
print(buy(day,'D',account,1))
print(account)
print(sell(day,'B',account,6))
print(account)
print(sell(day,'C',account,3))
print(account)
day = 2
print(f"day = {day}")
print(sell(day,'A',account,3))
print(account)
print(buy(day, 'B',account,2))
print(account)
```

Here is the output:

```
day = 0
{'fund': 1000}
('A', 3, 200, 600)
{'fund': 400, 'A': 3}
('B', 4, 20, 80)
{'fund': 320, 'A': 3, 'B': 4}
('B', 1, 20, 20)
{'fund': 300, 'A': 3, 'B': 5}
False
{'fund': 300, 'A': 3, 'B': 5}
day = 1
('A', 2, 190, 380)
{'fund': 680, 'A': 1, 'B': 5}
('D', 1, 91, 91)
{'fund': 589, 'A': 1, 'B': 5, 'D': 1}
('B', 5, 25, 125)
{'fund': 714, 'A': 1, 'D': 1}
False
{'fund': 714, 'A': 1, 'D': 1}
day = 2
('A', 1, 195, 195)
{'fund': 909, 'D': 1}
('B', 2, 30, 60)
{'fund': 849, 'D': 1, 'B': 2}
```

On day 0, the user purchased three shares of A costing $200/share for a total of $600. The account reflects is {'fund': 400, 'A': 3}. The user then purchased four shares of B, then one share of B. The account was then {'fund': 300, 'A': 3, 'B': 5}. The user then tried to buy three shares of D, but that costs $360 so the transaction was declined returning false. On day 1, the user sold two shares of A, but its value changed from $200 to $190 so the account fund was {'fund': 680, 'A': 1, 'B': 5} since $300 + 2($190) = $680. We point out that day the user sold six shares of B. The user only owned five, but this transaction is interpreted as selling all shares. The value was added to the fund (after purchasing one share of D). Observe that the account is {'fund': 714, 'A': 1, 'D': 1}, so the key for B was removed, since no shares were owned.

Deliverables Programming Problem 5

- Complete the two functions.
- Since account is call-by-reference, it will be changed locally in the function call.
- *Advice:* Implement buy first and correctly, then implement sell. Sketch out the steps on paper (or as a comment) then work on that rather than simply typing away.
- There are several ways to delete a key to remind you. Here's one:

```
>>> x = {}
>>> x['1'] = "Spring Break"
>>> x
{'1': 'Spring Break'}
>>> x['2'] = "Walking Kaiser & Ursala"
>>> x
{'1': 'Spring Break', '2': 'Walking Kaiser & Ursala'}
>>> del x['1']
>>> x
{'2': 'Walking Kaiser & Ursala'}
```

6: Optimus Prime

Consider some $p \in \mathbb{Z}$ where $p > 1$. If the only factors are $p, 1$, then p is called a *prime number*. If p is not a prime number, then it is called a *composite number*. Remember, a factor is a number that integer divides another. Prime numbers are important to cryptography and have fascinated people for a long time. Let's leverage modulo to find a small list of primes. Using modulo, implement a Boolean function prime(p) that returns true if p is prime for any positive integer p > 1.

Here is code:

```
ps = []
for p in range(2,100):
    if prime(p):
        ps.append(p)

print(ps)
```

and its behavior:

```
[2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47, 53, 59, 61, ↩
    67, 71, 73, 79, 83, 89, 97]
```

Deliverables Programming Problem 6

- Complete the prime function.
- Given a number p, it's prime if no other number $p-1, p-2, \ldots, 2$ is a factor. Do a little mathematics and reduce the size of the numbers you have to check.
- You are only allowed to use modulo and integer division if at all.

Programming partners

adeglur@iu.edu, surehma@iu.edu
chenkay@iu.edu, lomwean@iu.edu
aydclink@iu.edu, zstopher@iu.edu
achitred@iu.edu, trimedio@iu.edu
rachguo@iu.edu, snlnu@iu.edu
lineri@iu.edu, jmatczak@iu.edu
abelani@iu.edu, joebrahm@iu.edu
emiellis@iu.edu, nmetlapa@iu.edu
kimyouh@iu.edu, corrosar@iu.edu
forlin@iu.edu, andmuchi@iu.edu
ankotel@iu.edu, mitcchar@iu.edu
balian@iu.edu, frvisco@iu.edu
njustin@iu.edu, tiffyou@iu.edu
jailjohn@iu.edu, traepena@iu.edu
allennam@iu.edu, wangjinz@iu.edu
cfahy@iu.edu, arstover@iu.edu
careyjo@iu.edu, wagnebr@iu.edu
jhbruce@iu.edu, bfmark@iu.edu
okorkmaz@iu.edu, sparab@iu.edu
sambereb@iu.edu, jlopezmo@iu.edu
tkalale@iu.edu, weinstab@iu.edu
sebcont@iu.edu, archshar@iu.edu
cjduffie@iu.edu, camyee@iu.edu
andhodge@iu.edu, msolomie@iu.edu
sgudapur@iu.edu, nrimawi@iu.edu
nkhai@iu.edu, saravula@iu.edu
cdinman@iu.edu, peterabl@iu.edu
craftdc@iu.edu, rimaeda@iu.edu
ecabebe@iu.edu, crschuit@iu.edu
jdemian@iu.edu, rahkpate@iu.edu
arkali@iu.edu, wardjohn@iu.edu
cbremner@iu.edu, masoroki@iu.edu
zaycarey@iu.edu, shmulla@iu.edu
blberger@iu.edu, brshelby@iu.edu
nabatja@iu.edu, stewti@iu.edu
kespejo@iu.edu, amtsmith@iu.edu
ecanant@iu.edu, cmm23@iu.edu
lrborows@iu.edu, adpulipa@iu.edu
chamblia@iu.edu, mvsimi@iu.edu

shbivens@iu.edu, nicmill@iu.edu
ocrowder@iu.edu, manushah@iu.edu
allentp@iu.edu, xsoto@iu.edu
akumares@iu.edu, alexpan@iu.edu
lasdodd@iu.edu, cmoverfe@iu.edu
arkhal@iu.edu, avaweish@iu.edu
tkonkim@iu.edu, menonvi@iu.edu
cgreywal@iu.edu, breramer@iu.edu
benitezb@iu.edu, bsai@iu.edu
ejbusch@iu.edu, benserna@iu.edu
wchorich@iu.edu, ntorpoco@iu.edu
lbadredd@iu.edu, colbstap@iu.edu
cedonica@iu.edu, msantell@iu.edu
lawlin@iu.edu, ishetty@iu.edu
adevill@iu.edu, ymarrakc@iu.edu
reychaud@iu.edu, nvoneper@iu.edu
anudivek@iu.edu, smunikun@iu.edu
changali@iu.edu, ashahi@iu.edu
bcabral@iu.edu, haoztian@iu.edu
burkhel@iu.edu, gstogdil@iu.edu
madimond@iu.edu, gravipat@iu.edu
jaxkatz@iu.edu, ceprenti@iu.edu
reafrank@iu.edu, leoett@iu.edu
auconner@iu.edu, chsabir@iu.edu
shaagore@iu.edu, katepate@iu.edu
nbkotur@iu.edu, nsenthi@iu.edu
renahan@iu.edu, jamread@iu.edu
anchinna@iu.edu, nsmurthy@iu.edu
zcarmich@iu.edu, peterant@iu.edu
daveanu@iu.edu, pratmodi@iu.edu
chodgman@iu.edu, auspiege@iu.edu
langgrav@iu.edu, bwmoore@iu.edu
harmonnc@iu.edu, ctsvetan@iu.edu
conkeen@iu.edu, javitto@iu.edu
logellis@iu.edu, aquintas@iu.edu
agunde@iu.edu, syeshala@iu.edu
boding@iu.edu, vupatel@iu.edu
rcerauli@iu.edu, iopfings@iu.edu
nkushnir@iu.edu, mgstack@iu.edu
markdeng@iu.edu, reddysk@iu.edu
askaul@iu.edu, abbalokh@iu.edu

pafrost@iu.edu, twirges@iu.edu
ykassahu@iu.edu, etmillho@iu.edu
jkchaura@iu.edu, arangi@iu.edu
jamlian@iu.edu, khwillar@iu.edu
zlangley@iu.edu, tmukund@iu.edu
sdharmmi@iu.edu, bpovinel@iu.edu
sjbarry@iu.edu, jocpoiri@iu.edu
cc96@iu.edu, camlueck@iu.edu
jabbarke@iu.edu, ggriorda@iu.edu
ml82@iu.edu, jusshah@iu.edu
mgrass@iu.edu, jianlou@iu.edu
ryandaun@iu.edu, amatariy@iu.edu
aadkhan@iu.edu, rsreedh@iu.edu
marbelli@iu.edu, teswank@iu.edu
beckab@iu.edu, ap59@iu.edu
samjimen@iu.edu, mxrobins@iu.edu
belleli@iu.edu, mackeyr@iu.edu
gmhowell@iu.edu, jomalek@iu.edu
mbadwe@iu.edu, kammich@iu.edu
pkurzawa@iu.edu, johnsanc@iu.edu
jakedeck@iu.edu, olipfing@iu.edu
jvgottum@iu.edu, ksreejit@iu.edu
presharr@iu.edu, animerch@iu.edu
harrisdg@iu.edu, tryerrag@iu.edu
blevidut@iu.edu, joprayit@iu.edu
hedavid@iu.edu, shubtiwa@iu.edu
nsfried@iu.edu, smremmer@iu.edu
amgandhi@iu.edu, wmccarte@iu.edu
camehugh@iu.edu, njvinson@iu.edu
berrynm@iu.edu, ptapiaro@iu.edu